

# Finals Task 1. Data Preparations and Analysis using SQL

In this project, I built a SQL analytics script to clean and standardize raw call center timestamps, applied descriptive aggregation and window functions to evaluate peak durations and SLA compliance across locations, and broke down customer volume by day of the week to uncover key operational trends.

In this image, I performed the data cleaning phase by converting the text-based `call_timestamp` field into a standard SQL date format using the `str_to_date()` function after temporarily disabling safe update settings.

The screenshot displays the MySQL Workbench interface. The SQL editor contains a script to update the `calls` table, converting the `call_timestamp` field to a standard date format using the `str_to_date()` function. The script includes comments and a `LIMIT 10` clause for testing.

```
1 call_timestamp CHAR(10),
2 reason CHAR(20),
3 city CHAR(20),
4 state CHAR(20),
5 channel CHAR(20),
6 response_time CHAR(20),
7 call_duration_minutes INT,
8 call_center CHAR(20)
9 );
10
11 SET SQL_SAFE_UPDATES = 0;
12 UPDATE calls SET call_timestamp = str_to_date(call_timestamp, "%m/%d/%Y");
13 SET SQL_SAFE_UPDATES = 1;
14 SELECT * FROM calls LIMIT 10;
```

The Results Grid shows the first 10 rows of the `calls` table, displaying columns: ID, cust\_name, sentiment, csat\_score, call\_timestamp, reason, city, state, channel, response\_time, call\_duration\_minutes, and call\_center.

| ID                        | cust_name             | sentiment     | csat_score | call_timestamp | reason           | city           | state          | channel     | response_time | call_duration_minutes | call_center    |
|---------------------------|-----------------------|---------------|------------|----------------|------------------|----------------|----------------|-------------|---------------|-----------------------|----------------|
| DNK-5707809-w-055481-RJ   | Anahie Gardner        | Neutral       | 7          | 10/29/2020     | Billing Question | Detroit        | Michigan       | Call Center | Within SLA    | 17                    | Los Angeles/CA |
| QKQ-72228678-w-1021294-YT | Christina Kibbey      | Very Positive | 0          | 10/05/2020     | Service Outage   | Spartanburg    | South Carolina | ChatBot     | Within SLA    | 23                    | Baltimore/MD   |
| 077-2002932-A-920315-D    | Averil Brundett       | Negative      | 0          | 10/04/2020     | Billing Question | Gainesville    | Florida        | Call Center | Above SLA     | 45                    | Los Angeles/CA |
| ZZB-96807559-4-02008-w-07 | Noreen Laffina        | Very Negative | 1          | 10/17/2020     | Billing Question | Portland       | Oregon         | ChatBot     | Within SLA    | 12                    | Los Angeles/CA |
| 00U-69461719-O-10482-Fm   | Tama van der Beken    | Very Positive | 0          | 10/17/2020     | Payment          | Portland       | Indiana        | Call Center | Within SLA    | 23                    | Los Angeles/CA |
| 315-7972660-A-22035-Aq    | Karlyn Dolan          | Neutral       | 5          | 10/20/2020     | Billing Question | Salt Lake City | Utah           | Call Center | Within SLA    | 25                    | Baltimore/MD   |
| AZJ-0505097-A-10542-PT    | Philip Dowling        | Neutral       | 8          | 10/16/2020     | Billing Question | Tyler          | Texas          | ChatBot     | Within SLA    | 31                    | Baltimore/MD   |
| TWV-27007918-I-608785-Xw  | Krista de Tasqueville | Positive      | 0          | 10/21/2020     | Billing Question | New York City  | New York       | ChatBot     | Below SLA     | 37                    | Los Angeles/CA |
| 0SG-4450118-P-344472-ZJ   | Owen Libery           | Very Negative | 0          | 10/02/2020     | Billing Question | Dallas         | Texas          | Deal        | Below SLA     | 37                    | Baltimore/MD   |
| RLC-641082072-2-285141-VS | Port Bragg            | Neutral       | 0          | 10/07/2020     | Billing Question | Cincinnati     | Ohio           | ChatBot     | Within SLA    | 12                    | Baltimore/MD   |

The Output tab shows the execution of the SQL script, including the `UPDATE` statement and the `SELECT` statement, with the number of rows affected and the duration of each operation.

In this image, I tracked peak call lengths over time by applying the `MAX() OVER (PARTITION BY ...)` window function to calculate the maximum call duration for each unique timestamp.

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
62
63 SELECT call_center, response_time, COUNT(*) AS count
64 FROM calls group by 1,2 order by 1,3 DESC;
65
66
67
68
69 SELECT call_timestamp, MAX(call_duration_minutes) OVER(partition by call_timestamp) AS max_call_duration FROM calls GROUP BY 1 ORDER BY 2 DESC;
```

The Results grid displays the following data:

| call_timestamp | max_call_duration |
|----------------|-------------------|
| 2020-10-04     | 45                |
| 2020-10-22     | 45                |
| 2020-10-26     | 41                |
| 2020-10-13     | 39                |
| 2020-10-30     | 37                |
| 2020-10-21     | 37                |
| 2020-10-14     | 37                |
| 2020-10-03     | 37                |
| 2020-10-12     | 35                |
| 2020-10-09     | 35                |
| 2020-10-25     | 34                |
| 2020-10-16     | 31                |
| 2020-10-20     | 30                |
| 2020-10-02     | 30                |
| 2020-10-19     | 30                |
| 2020-10-18     | 28                |
| 2020-10-08     | 27                |
| 2020-10-27     | 26                |
| 2020-10-28     | 25                |
| 2020-10-05     | 23                |
| 2020-10-06     | 22                |
| 2020-10-24     | 22                |

The Output pane shows the execution of the query, with the following messages:

- 54 00:12:22 SELECT channel, count(), ROUND(COUNT(\*) / (SELECT COUNT(\*) FROM calls) \* 100, 1) AS pct FROM calls GROUP BY 1 ORDER BY 3 DESC LIMIT 0, 1000 4 rows(s) returned
- 55 00:12:22 SELECT response\_time, count(), ROUND(COUNT(\*) / (SELECT COUNT(\*) FROM calls) \* 100, 1) AS pct FROM calls GROUP BY 1 ORDER BY 3 DESC LIMIT 0, 1000 3 rows(s) returned
- 56 00:12:22 SELECT call\_center, count(), ROUND(COUNT(\*) / (SELECT COUNT(\*) FROM calls) \* 100, 1) AS pct FROM calls GROUP BY 1 ORDER BY 3 DESC LIMIT 0, 1000 4 rows(s) returned
- 57 00:12:22 SELECT state, COUNT(\*) FROM calls GROUP BY 1 ORDER BY 2 DESC LIMIT 0, 1000 51 rows(s) returned
- 58 00:13:17 SELECT call\_center, response\_time, COUNT(\*) AS count FROM calls group by 1,2 order by 1,3 DESC LIMIT 0, 1000 12 rows(s) returned
- 59 00:14:54 SELECT call\_timestamp, MAX(call\_duration\_minutes) OVER(partition by call\_timestamp) AS max\_call\_duration FROM calls GROUP BY 1 ORDER BY 2 DESC LIMIT 0, 1000 31 rows(s) returned

In this image, I generated a breakdown of service level agreement compliance across different locations, counting how many calls fell within, below, or above SLAs for each individual call center.

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
55 FROM calls GROUP BY 1 ORDER BY 3 DESC;
56
57 SELECT call_center, count(*), ROUND(COUNT(*) / (SELECT COUNT(*) FROM calls) * 100, 1) AS pct
58 FROM calls GROUP BY 1 ORDER BY 3 DESC;
59
60 SELECT state, COUNT(*) FROM calls GROUP BY 1 ORDER BY 2 DESC;
61
62
63 SELECT call_center, response_time, COUNT(*) AS count
64 FROM calls group by 1,2 order by 1,3 DESC;
```

The Results grid displays the following data:

| call_center   | response_time | count |
|---------------|---------------|-------|
| BaltimoreMD   | Within SLA    | 6025  |
| BaltimoreMD   | Below SLA     | 2768  |
| BaltimoreMD   | Above SLA     | 1389  |
| ChicagoIL     | Within SLA    | 1361  |
| ChicagoIL     | Below SLA     | 1361  |
| ChicagoIL     | Above SLA     | 697   |
| DenverCO      | Within SLA    | 1741  |
| DenverCO      | Below SLA     | 692   |
| DenverCO      | Above SLA     | 343   |
| Los AngelesCA | Within SLA    | 8668  |
| Los AngelesCA | Below SLA     | 2327  |
| Los AngelesCA | Above SLA     | 1729  |

The Output pane shows the execution of the query, with the following messages:

- 55 00:12:22 SELECT channel, count(), ROUND(COUNT(\*) / (SELECT COUNT(\*) FROM calls) \* 100, 1) AS pct FROM calls GROUP BY 1 ORDER BY 3 DESC LIMIT 0, 1000 4 rows(s) returned
- 56 00:12:22 SELECT response\_time, count(), ROUND(COUNT(\*) / (SELECT COUNT(\*) FROM calls) \* 100, 1) AS pct FROM calls GROUP BY 1 ORDER BY 3 DESC LIMIT 0, 1000 3 rows(s) returned
- 57 00:12:22 SELECT call\_center, count(), ROUND(COUNT(\*) / (SELECT COUNT(\*) FROM calls) \* 100, 1) AS pct FROM calls GROUP BY 1 ORDER BY 3 DESC LIMIT 0, 1000 4 rows(s) returned
- 58 00:12:22 SELECT state, COUNT(\*) FROM calls GROUP BY 1 ORDER BY 2 DESC LIMIT 0, 1000 51 rows(s) returned
- 59 00:13:17 SELECT call\_center, response\_time, COUNT(\*) AS count FROM calls group by 1,2 order by 1,3 DESC LIMIT 0, 1000 12 rows(s) returned
- 60 00:14:54 SELECT call\_timestamp, MAX(call\_duration\_minutes) OVER(partition by call\_timestamp) AS max\_call\_duration FROM calls GROUP BY 1 ORDER BY 2 DESC LIMIT 0, 1000 31 rows(s) returned
- 61 00:16:08 SELECT call\_center, response\_time, COUNT(\*) AS count FROM calls group by 1,2 order by 1,3 DESC LIMIT 0, 1000 12 rows(s) returned

In this image, I calculated high-level descriptive metrics for the dataset, revealing the minimum, maximum, and overall average call duration alongside a series of my exploratory diagnostic queries.

The screenshot shows a MySQL Workbench interface with a series of SQL queries in the 'Query 1' tab. The queries are as follows:

```

82 • SELECT MIN(call_duration_minutes) AS min_call_duration, MAX(call_duration_minutes) AS max_call_duration, AVG(call_duration_minutes) AS avg_call_duration
    FROM calls;
83
84
85 • SELECT call_center, response_time, COUNT(*) AS count
    FROM calls GROUP BY 1, 2 ORDER BY 1, 3 DESC;
86
87
88 • SELECT call_center, AVG(call_duration_minutes) FROM calls GROUP BY 1 ORDER BY 2 DESC;
89
90 • SELECT channel, AVG(call_duration_minutes) FROM calls GROUP BY 1 ORDER BY 2 DESC;
91
92 • SELECT state, COUNT(*) FROM calls GROUP BY 1 ORDER BY 2 DESC;
93
94 • SELECT state, reason, COUNT(*) FROM calls GROUP BY 1, 2 ORDER BY 1, 2, 3 DESC;
95
96 • SELECT state, sentiment, COUNT(*) FROM calls GROUP BY 1, 2 ORDER BY 1, 3 DESC;
97
98 • SELECT state, AVG(csat_score) AS avg_csat_score FROM calls WHERE csat_score != 0 GROUP BY 1 ORDER BY 2 DESC;
99
100 • SELECT sentiment, AVG(call_duration_minutes) FROM calls GROUP BY 1 ORDER BY 2 DESC;
  
```

The results are displayed in a table with the following columns: min\_call\_duration, max\_call\_duration, avg\_call\_duration. The results are:

| min_call_duration | max_call_duration | avg_call_duration |
|-------------------|-------------------|-------------------|
| 15                | 45                | 25.0212           |

The bottom of the screenshot shows the 'Output' tab with a table of results for the queries. The table has columns: #, Time, Action, Message, and Duration / Fetch. The results are as follows:

| #   | Time     | Action                                                                                                                    | Message             | Duration / Fetch      |
|-----|----------|---------------------------------------------------------------------------------------------------------------------------|---------------------|-----------------------|
| 102 | 00:22:20 | SELECT channel, AVG(call_duration_minutes) FROM calls GROUP BY 1 ORDER BY 2 DESC LIMIT 0, 1000                            | 4 row(s) returned   | 0.062 sec / 0.000 sec |
| 103 | 00:22:20 | SELECT state, COUNT(*) FROM calls GROUP BY 1 ORDER BY 2 DESC LIMIT 0, 1000                                                | 51 row(s) returned  | 0.047 sec / 0.000 sec |
| 104 | 00:22:20 | SELECT state, reason, COUNT(*) FROM calls GROUP BY 1, 2 ORDER BY 1, 2, 3 DESC LIMIT 0, 1000                               | 153 row(s) returned | 0.062 sec / 0.000 sec |
| 105 | 00:22:20 | SELECT state, sentiment, COUNT(*) FROM calls GROUP BY 1, 2 ORDER BY 1, 3 DESC LIMIT 0, 1000                               | 250 row(s) returned | 0.070 sec / 0.000 sec |
| 106 | 00:22:20 | SELECT state, AVG(csat_score) AS avg_csat_score FROM calls WHERE csat_score != 0 GROUP BY 1 ORDER BY 2 DESC LIMIT 0, 1000 | 51 row(s) returned  | 0.032 sec / 0.000 sec |
| 107 | 00:22:20 | SELECT sentiment, AVG(call_duration_minutes) FROM calls GROUP BY 1 ORDER BY 2 DESC LIMIT 0, 1000                          | 5 row(s) returned   | 0.047 sec / 0.000 sec |

In this image, I extracted the day name from the timestamp column to calculate the total volume of calls handled per day of the week, which revealed to me that Friday was the busiest day.

The screenshot shows a MySQL Workbench interface with a series of SQL queries in the 'Query 1' tab. The queries are as follows:

```

37 • SELECT MIN(call_timestamp) AS earliest_date, MAX(call_timestamp) AS most_recent FROM calls;
38 • SELECT MIN(call_duration_minutes) AS min_call_duration, MAX(call_duration_minutes) AS max_call_duration, AVG(call_duration_minutes) AS avg_call_duration FROM calls;
39 • SELECT call_center, response_time, COUNT(*) AS count
    FROM calls GROUP BY 1, 2 ORDER BY 1, 3 DESC;
40
41 • SELECT call_center, AVG(call_duration_minutes) FROM calls GROUP BY 1 ORDER BY 2 DESC;
42
43 • SELECT channel, AVG(call_duration_minutes) FROM calls GROUP BY 1 ORDER BY 2 DESC;
44
45 • SELECT state, COUNT(*) FROM calls GROUP BY 1 ORDER BY 2 DESC;
46
47 • SELECT state, reason, COUNT(*) FROM calls GROUP BY 1, 2 ORDER BY 1, 2, 3 DESC;
48
49 • SELECT state, sentiment, COUNT(*) FROM calls GROUP BY 1, 2 ORDER BY 1, 3 DESC;
46 • SELECT state, AVG(csat_score) AS avg_csat_score FROM calls WHERE csat_score != 0 GROUP BY 1 ORDER BY 2 DESC;
47 • SELECT sentiment, AVG(call_duration_minutes) FROM calls GROUP BY 1 ORDER BY 2 DESC;
48
49 • SELECT DAYNAME(call_timestamp) AS day_of_call, COUNT(*) AS num_of_calls
    FROM calls
    GROUP BY 1
    ORDER BY 2 DESC;
53
  
```

The results are displayed in a table with the following columns: day\_of\_call, num\_of\_calls. The results are:

| day_of_call | num_of_calls |
|-------------|--------------|
| Friday      | 5570         |
| Thursday    | 5491         |
| Wednesday   | 4449         |
| Tuesday     | 4408         |
| Saturday    | 4403         |
| Monday      | 4324         |
| Sunday      | 4296         |

The bottom of the screenshot shows the 'Output' tab with a table of results for the queries. The table has columns: #, Time, Action, Message, and Duration / Fetch. The results are as follows:

| #   | Time     | Action                                                                                                                      | Message             | Duration / Fetch      |
|-----|----------|-----------------------------------------------------------------------------------------------------------------------------|---------------------|-----------------------|
| 141 | 02:44:22 | SELECT state, reason, COUNT(*) FROM calls GROUP BY 1, 2 ORDER BY 1, 2, 3 DESC LIMIT 0, 1000                                 | 153 row(s) returned | 0.070 sec / 0.000 sec |
| 142 | 02:44:22 | SELECT state, sentiment, COUNT(*) FROM calls GROUP BY 1, 2 ORDER BY 1, 3 DESC LIMIT 0, 1000                                 | 250 row(s) returned | 0.062 sec / 0.000 sec |
| 143 | 02:44:22 | SELECT state, AVG(csat_score) AS avg_csat_score FROM calls WHERE csat_score != 0 GROUP BY 1 ORDER BY 2 DESC LIMIT 0, 1000   | 51 row(s) returned  | 0.031 sec / 0.000 sec |
| 144 | 02:44:22 | SELECT sentiment, AVG(call_duration_minutes) FROM calls GROUP BY 1 ORDER BY 2 DESC LIMIT 0, 1000                            | 5 row(s) returned   | 0.047 sec / 0.000 sec |
| 145 | 02:45:37 | SELECT call_timestamp FROM calls LIMIT 5                                                                                    | 5 row(s) returned   | 0.000 sec / 0.000 sec |
| 146 | 02:45:50 | SELECT DAYNAME(call_timestamp) AS day_of_call, COUNT(*) AS num_of_calls FROM calls GROUP BY 1 ORDER BY 2 DESC LIMIT 0, 1000 | 7 row(s) returned   | 0.031 sec / 0.000 sec |